Module 12 - Threading

Table of Contents

- [ ] 1. Why Threading Matters in Cybersecurity
- [ ] 2. Linear vs Parallel — The Blocking Problem
- [ ] 3. Your First Thread — `Thread`, `start()`, `run()`
- [ ] 4. `join()` — Waiting for Threads to Finish
- [ ] 5. Sharing Data Between Threads — The Race Condition
- [ ] 6. `Lock` and `RLock` — Protecting Shared Resources
- [ ] 7. When to Use Threading (and When Not to)
- [ ] 8. Summary and Next Steps

---

1. Why Threading Matters in Cybersecurity

Most security tools spend most of their time **waiting** — waiting for a network connection to respond, waiting for a file to be read from disk, waiting for a server to reply. This waiting time is wasted if you can only do one thing at a time.

Threading solves this by letting your program do multiple things at once.

Without threading	With threading
Scan 1,000 ports one by one — 5 minutes	Scan 1,000 ports in parallel — 10 seconds
Read 10 log files one by one — slow	Read all 10 simultaneously — fast
Check 100 IPs against threat intel — sequential	Check all 100 at once — fast

The `data/` folder for this notebook

We have 10 server log files — one per host in our infrastructure:

```
data/
  web-01.log    web-02.log    db-01.log     api-01.log    api-
02.log
  mail-01.log   vpn-01.log    proxy-01.log  admin-01.log  backup-
01.log
```

Each file contains 12 security event lines for that host. We will use threading to read all 10 files at the same time.

---

2. Linear vs Parallel — The Blocking Problem

Think of **linear programming** like a single security analyst who must finish reviewing each host's logs completely before moving on to the next.

Think of **parallel programming** like a SOC team where each analyst takes one host simultaneously — the whole review finishes in the time it takes to review one host.

In Python, the default execution model is **linear**: one instruction completes before the next one starts. This is called **blocking** — the program blocks and waits at each step.

The `time.sleep()` call is the simplest demonstration: it freezes the entire program.

In [ ]:
```
# Topic: blocking vs non-blocking — timing a linear scan simulation
# Simulates what happens when you connect to each of 5 hosts one at a time

import time

hosts_to_check = ["web-01", "web-02", "db-01", "api-01", "api-02"]

def check_host_linear(hostname):
    """Simulates reading a host's log — the sleep represents IO wait time."""
    print(f"  Checking {hostname}...")
    time.sleep(1)   # represents: connecting, reading, processing
    print(f"  Done:     {hostname}")


print("=== LINEAR (sequential) ===")
start = time.time()

for host in hosts_to_check:
    check_host_linear(host)

elapsed = time.time() - start
print(f"\nTotal time (linear): {elapsed:.1f}s  — one host at a time")
```

In [ ]:
```
# Topic: the same task, now with threading — timing comparison

import time
from threading import Thread


def check_host_threaded(hostname):
    """Same function — but called from a thread."""
    print(f"  Checking {hostname}...")
    time.sleep(1)   # each thread sleeps independently — they don't block each othe
    print(f"  Done:     {hostname}")
```

```
print("=== THREADED (parallel) ===")
start = time.time()

threads = []
for host in hosts_to_check:
    t = Thread(target=check_host_threaded, args=(host,))
    threads.append(t)
    t.start()          # start all threads — they run simultaneously

for t in threads:
    t.join()           # wait for ALL threads to finish before continuing

elapsed = time.time() - start
print(f"\nTotal time (threaded): {elapsed:.1f}s  — all hosts at the same time")
```

Notice the output order is **non-deterministic** — threads don't finish in the order they start. This is normal and is one of the key things to manage when threading.

The timing difference: **5 seconds vs ~1 second** — that scales massively. A port scanner checking 1,000 ports with a 1-second timeout would take **16 minutes** linearly but under **2 seconds** with 1,000 threads.

---

3. Your First Thread — `Thread`, `start()`, `run()`

There are two ways to create a thread in Python. Both are valid — choose based on context.

Method 1: `Thread(target=function, args=(...))`

The simplest approach — pass a function and its arguments directly.

```
from threading import Thread

t = Thread(target=my_function, args=(arg1, arg2))
t.start()    # begin execution in a new thread
```

Method 2: Subclassing `Thread`

More structured — define a class that inherits from `Thread`. Required when the thread needs to hold state (attributes) or do complex setup.

```
from threading import Thread

class MyThread(Thread):
    def __init__(self, arg1):
        Thread.__init__(self)   # MUST call parent __init__
        self.arg1 = arg1
```

```
    def run(self):              # MUST be named 'run' — called by start()
        # thread logic here
        pass

t = MyThread(arg1="value")
t.start()    # calls run() in a new thread
```

Key rule: never call `run()` directly

```
t.run()    # WRONG — runs the code in the CURRENT thread, no parallelism
t.start()  # CORRECT — launches the code in a NEW thread
```

In [ ]:

```
# Topic: Method 1 — target= with args — reading one log file per thread

import time
from threading import Thread


def read_host_log(hostname):
    """Read a host's log file and print a summary — called from a thread."""
    log_path = f"data/{hostname}.log"
    event_count = 0

    with open(log_path, "r") as f:
        for line in f:
            if line.strip():
                event_count += 1

    print(f"  [{hostname}] {event_count} events logged")


hosts = ["web-01", "web-02", "db-01"]

threads = []
for host in hosts:
    t = Thread(target=read_host_log, args=(host,))  # args must be a tuple
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("All log reads complete.")
```

In [ ]:

```
# Topic: Method 2 — subclassing Thread — with stored results
# Use when the thread needs to hold its result for later retrieval

import time
from threading import Thread


class LogReaderThread(Thread):
    def __init__(self, hostname):
        Thread.__init__(self)           # always call parent __init__
```

```
        self.hostname = hostname        # store arguments as attributes
        self.event_count = 0            # attribute to hold the result
        self.critical_events = []       # collect critical events found

    def run(self):                      # called by start() in a new thread
        log_path = f"data/{self.hostname}.log"

        with open(log_path, "r") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                self.event_count += 1
                # Flag brute-force successes as critical
                if "Brute force succeeded" in line:
                    self.critical_events.append(line)


hosts = ["web-01", "api-01", "admin-01"]

# Create and start all threads
readers = [LogReaderThread(h) for h in hosts]
for r in readers:
    r.start()

# Wait for all to finish, then retrieve results
for r in readers:
    r.join()
    print(f"[{r.hostname}] {r.event_count} events | {len(r.critical_events)} critic
    for evt in r.critical_events:
        print(f"    !! {evt}")
```

🎯 Practice — First Threads

Exercise 1 — Write

Using `Thread(target=..., args=(...))`, write a function `count_failed_logins(hostname)` that opens `data/{hostname}.log`, counts lines containing `SSH_LOGIN_FAILED`, and prints: `[hostname] N failed login attempts`

Launch one thread per host for all 10 hosts in `data/`. Wait for all threads to finish.

In [ ]:
```
# Your code here
import os
from threading import Thread
```

▶ 💡 **SOLUTION**

Exercise 2 — Predict

Before running the cell below, predict:

1. Will "Scan launched" print before or after the thread output?

2. Will the hosts always print in the same order?
3. What happens if you remove the `t.join()` calls?

In [ ]:
```
# Predict first, then run!
import time
from threading import Thread

def slow_check(hostname):
    time.sleep(0.2)
    print(f"  Done: {hostname}")

threads = [
    Thread(target=slow_check, args=("web-01",)),
    Thread(target=slow_check, args=("db-01",)),
    Thread(target=slow_check, args=("api-01",)),
]
for t in threads:
    t.start()

print("Scan launched")     # Does this print before or after the thread output?

for t in threads:
    t.join()

print("All done")          # And this one?
```

---

4. `join()` — Waiting for Threads to Finish

When you call `t.start()`, the main program continues immediately — it does not wait for the thread. This means the code that follows `start()` can run before the thread has finished its work.

`t.join()` tells the main thread: "*stop here and wait until thread* `t` *is done.*"

The pattern you will use in every multi-threaded security tool:

```
threads = []
for item in items:
    t = Thread(target=process, args=(item,))
    threads.append(t)
    t.start()           # launch all threads first

for t in threads:       # then join all
    t.join()            # wait for each one — main continues when ALL are
done

# ← only reaches here when every thread has completed
print("All threads done")
```

Why join matters for result correctness

Without `join()`, your results list may be incomplete — threads still writing results while you're already reading and reporting them.

In [ ]:
```
# Topic: join() is critical for correctness — collecting results from all threads

import os
import time
from threading import Thread


class HostScanThread(Thread):
    def __init__(self, hostname):
        Thread.__init__(self)
        self.hostname = hostname
        self.intrusion_attempts = 0

    def run(self):
        with open(f"data/{self.hostname}.log", "r") as f:
            for line in f:
                if "INTRUSION_ATTEMPT" in line:
                    self.intrusion_attempts += 1


log_files = sorted(f for f in os.listdir("data") if f.endswith(".log"))
hostnames = [f.replace(".log", "") for f in log_files]

scanners = [HostScanThread(h) for h in hostnames]

start = time.time()
for s in scanners:
    s.start()

for s in scanners:       # join() ensures all threads finish before we read results
    s.join()

elapsed = time.time() - start

# Only reached here after ALL 10 threads have completed
total_intrusions = sum(s.intrusion_attempts for s in scanners)
print(f"Infrastructure intrusion scan complete in {elapsed:.2f}s")
print(f"Total intrusion attempts across all hosts: {total_intrusions}")
print()
for s in scanners:
    print(f"  {s.hostname}: {s.intrusion_attempts} intrusion attempt(s)")
```

🎯 Practice — `join()`

Exercise 3 — Fix

The code below has a bug: it reads `results` before the threads have finished writing to it. Fix it so the final count is always correct.

In [ ]:
```
# FIX the bug in this code:

import time
from threading import Thread

results = []

def find_port_scan_events(hostname):
    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "PORT_SCAN" in line:
                results.append(hostname)   # this append is not thread-safe — we'll

hosts = ["web-01", "web-02", "db-01", "api-01", "api-02"]
threads = [Thread(target=find_port_scan_events, args=(h,)) for h in hosts]

for t in threads:
    t.start()

# BUG: reading results HERE before threads finish
print(f"Hosts with port scan events: {len(results)}")  # likely 0 or wrong!

# Fix: add join() calls in the right place
```

▶ 💡 **SOLUTION**

---

5. Sharing Data Between Threads — The Race Condition

When multiple threads modify the same shared variable, you can get **unpredictable results**. This is called a **race condition**.

The analogy

Imagine two SOC analysts both updating the same incident ticket at the same time — the second analyst's save overwrites the first one's changes. One update is silently lost.

Why `counter += 1` is not safe in threads

Even a simple increment is actually **three separate operations** at the CPU level:

```
Step 1: READ   current value of counter (e.g. 42)
Step 2: ADD    1 to it              (42 + 1 = 43)
Step 3: WRITE  result back          (counter = 43)
```

If two threads both execute step 1 before either reaches step 3, both see `42`, both compute `43`, and both write `43` — but the counter should have become `44`. One increment is silently lost.

In [ ]:
```python
# Topic: demonstrating a race condition — a shared failed_login_count
# Run this several times — you'll see different (wrong) totals

import time
from threading import Thread

# Shared counter — dangerous when modified from multiple threads
failed_login_count = 0

def process_host_log(hostname):
    global failed_login_count
    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "SSH_LOGIN_FAILED" in line:
                # RACE CONDITION: three steps, threads can interleave
                failed_login_count += 1

import os
hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

threads = [Thread(target=process_host_log, args=(h,)) for h in hostnames]
for t in threads:
    t.start()
for t in threads:
    t.join()

# Expected: 10 hosts × 3 failed logins each = 30
# Actual: may be less due to race conditions
print(f"Failed logins counted: {failed_login_count}  (expected: 30)")
print("Run this cell multiple times — you may get different results each time.")
```

---

6. `Lock` and `RLock` — Protecting Shared Resources

A **Lock** is like a room with a single-entry door. Only one thread can be inside at a time. While one thread holds the lock, any other thread that tries to acquire it must wait outside.

In security terms: think of it as a mutex (mutual exclusion) — used in production security tools to protect shared log buffers, counters, and result lists.

Object	Use case
`threading.Lock()`	Simple lock — one thread at a time
`threading.RLock()`	Re-entrant lock — same thread can acquire it multiple times

Best practice: always use `with lock:` rather than `lock.acquire()` / `lock.release()` — the `with` statement ensures the lock is released even if an exception occurs.

```
from threading import Lock

lock = Lock()

# Safe:
with lock:
    shared_variable += 1   # only one thread executes this at a time
```

In [ ]:
```
# Topic: Lock — fixing the race condition in the failed login counter

import os
from threading import Thread, Lock

failed_login_count = 0
lock = Lock()   # one global lock protecting the shared counter


def process_host_log_safe(hostname):
    global failed_login_count
    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "SSH_LOGIN_FAILED" in line:
                with lock:                  # only one thread can increment at a ti
                    failed_login_count += 1


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

threads = [Thread(target=process_host_log_safe, args=(h,)) for h in hostnames]
for t in threads:
    t.start()
for t in threads:
    t.join()

# Now always correct — the lock prevents the race
print(f"Failed logins counted: {failed_login_count}  (expected: 30 — always correct
```

In [ ]:
```
# Topic: Lock protecting a shared list — aggregating critical events
# A list.append() is atomic in CPython but this pattern is the correct habit

import os
from threading import Thread, Lock

critical_events = []        # shared result list
events_lock = Lock()        # lock protecting it
```

```
def find_critical_events(hostname):
    """Collect all brute-force success events from one host."""
    local_events = []       # collect locally first — no lock needed for local var

    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "Brute force succeeded" in line:
                local_events.append(line.strip())

    # Only hold the lock for the final write — minimize lock time
    with events_lock:
        critical_events.extend(local_events)


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

threads = [Thread(target=find_critical_events, args=(h,)) for h in hostnames]
for t in threads:
    t.start()
for t in threads:
    t.join()

print(f"Critical events (brute force success) across all hosts: {len(critical_event
for evt in critical_events:
    print(f"  !! {evt}")
```

🎯 Practice — Race Conditions and Locks

Exercise 4 — Write

Write a threaded log scanner with a shared results dict `events_by_type`:

`events_by_type = {"SSH_LOGIN_FAILED": 0, "INTRUSION_ATTEMPT": 0, "PORT_SCAN": 0}`

Each thread processes one host's log file and increments the correct counter. Use a `Lock` to protect the shared dict. Launch one thread per host for all 10 hosts.

After joining, print the final counts. Expected: `SSH_LOGIN_FAILED: 30`, etc.

In [ ]:
```
# Your code here
import os
from threading import Thread, Lock
```

▶ 💡 **SOLUTION**

Exercise 5 — Write

Using `RLock` and the subclassing approach, write a `LogWriterThread` class that:

- Takes a `hostname` and a `shared_lock`

- Reads `data/{hostname}.log`
- Appends all `INTRUSION_ATTEMPT` lines to `data/intrusions.txt` using the lock to ensure writes don't interleave
- Include a header line per host: `=== Intrusions from {hostname} ===`

Launch 5 threads (first 5 hosts). Read `data/intrusions.txt` afterwards.

In [ ]:
```
# Your code here
from threading import Thread, RLock
```

▶ 💡 **SOLUTION**

---

7. When to Use Threading (and When Not to)

Threading is not always the right tool. Understanding when to use it is as important as knowing how to use it.

IO-bound tasks — threading HELPS

IO-bound means: most of the time is spent **waiting for input/output** — network, disk, database. The CPU is idle while waiting.

Threading shines here because multiple threads can be waiting simultaneously:

Security use case	IO-bound?	Threading useful?
Port scanner (connect to 1,000 ports)	Yes — waiting for TCP responses	**Yes**
Log ingestion (reading 50 log files)	Yes — waiting for disk reads	**Yes**
Threat intel lookup (HTTP to VirusTotal API)	Yes — waiting for HTTP responses	**Yes**
Password hash cracking	No — pure CPU computation	**No**
Parsing/transforming data in memory	No — pure CPU	**No**

CPU-bound tasks — use `multiprocessing` instead

Python has the **Global Interpreter Lock (GIL)** — a mechanism that prevents multiple threads from executing Python bytecode simultaneously.

This means for CPU-heavy tasks, threading gives you **no parallelism** — the GIL forces threads to take turns. For CPU-bound work, use `multiprocessing` or `concurrent.futures.ProcessPoolExecutor`.

```
IO-bound work  → threading.Thread  or
concurrent.futures.ThreadPoolExecutor
CPU-bound work → multiprocessing   or
concurrent.futures.ProcessPoolExecutor
```

The Advanced notebook covers `ThreadPoolExecutor` — the modern, cleaner way to manage thread pools for IO-bound security tools.

---

8. Summary and Next Steps

Key Rules

Rule	Why it matters
Use `t.start()` not `t.run()`	`run()` doesn't launch a new thread
Always `t.join()` before reading results	Results may be incomplete without it
Use `with lock:` not `acquire()` / `release()`	Lock always released even on exception
Hold the lock for the minimum time	Longer lock = more threads waiting = less parallelism
Collect results locally, write to shared once	Reduces time spent holding the lock
Use threading for IO-bound tasks only	CPU-bound tasks: use `multiprocessing`

What comes next

- **01_Advanced_Threading.ipynb** — `ThreadPoolExecutor` (cleaner pool management), thread-safe `Queue`, daemon threads, and a full threaded log ingestion pipeline